

Engenharia de Tokens: Autenticação Stateless com JWT

Manual de Operações Práticas e Testes de Penetração (Baseado na RFC 8725)

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c

1. Cabeçalho (Header)

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

Declaração do algoritmo de assinatura (HS256) e tipo do token (JWT). Codificado em Base64Url.

2. Carga Útil (Payload)

```
{  
  "sub": "1234567890",  
  "name": "John Doe",  
  "iat": 1516239022  
}
```

Contém as declarações (claims) sobre a entidade (usuário) e metadados. Codificado em Base64Url. Inclui 'sub' (subject), 'name' e 'iat' (issued at).

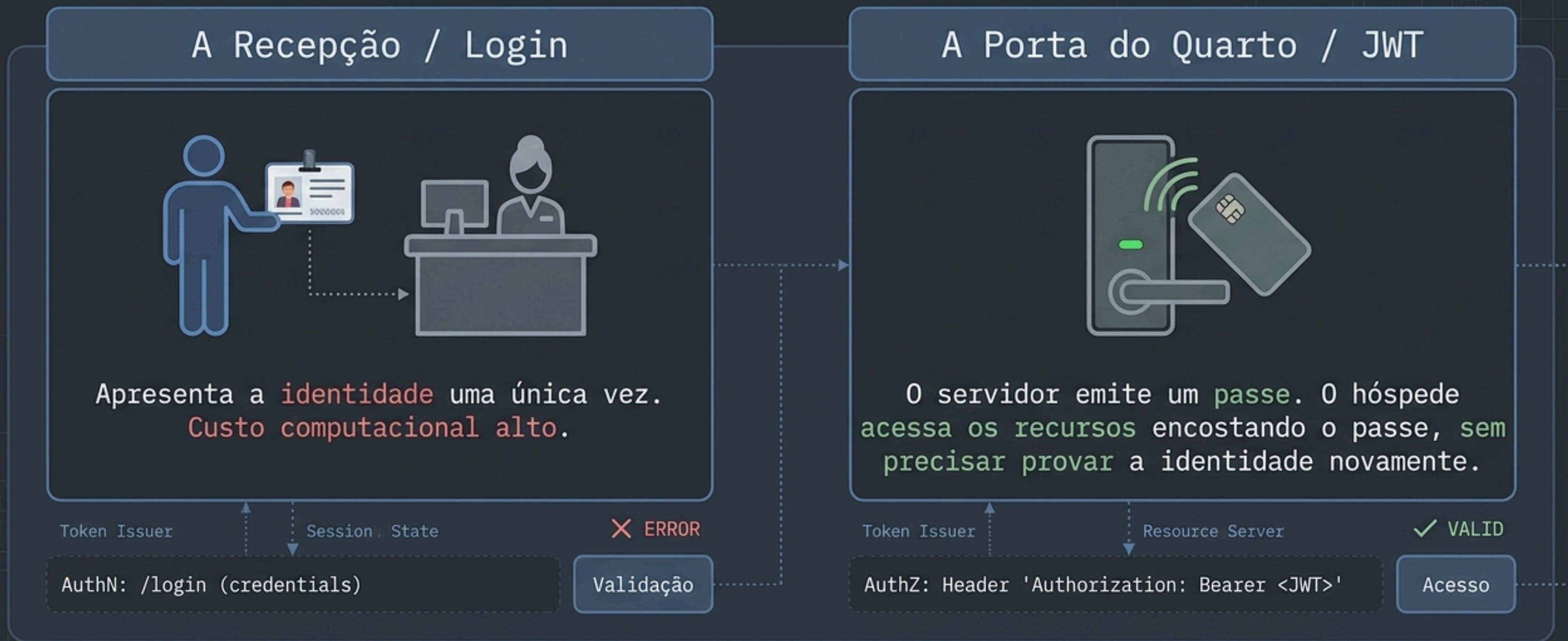
3. Assinatura (Signature)

```
HMACSHA256(  
  base64UrlEncode(header) + "." +  
  base64UrlEncode(payload),  
  secret  
)
```

Verifica a integridade do token e a autenticidade do emissor. Gerado utilizando o algoritmo declarado no cabeçalho e uma chave secreta. Essencial para evitar adulteração.

O Paradigma do Cartão Magnético

Diferenciando o Login (AuthN) da Sessão (AuthZ).



Raio-X: A Anatomia de um JWT

A regra inegociável: JSON Web Tokens são assinados, não criptografados.

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.

eyJpZCI6MSwidXN1YXJpbyI6ImImFkbWlwIn0.

Header

```
{  
  alg: HS256,  
  typ: JWT  
}
```

Payload

```
{  
  id: 1,  
  usuario: admin  
}
```



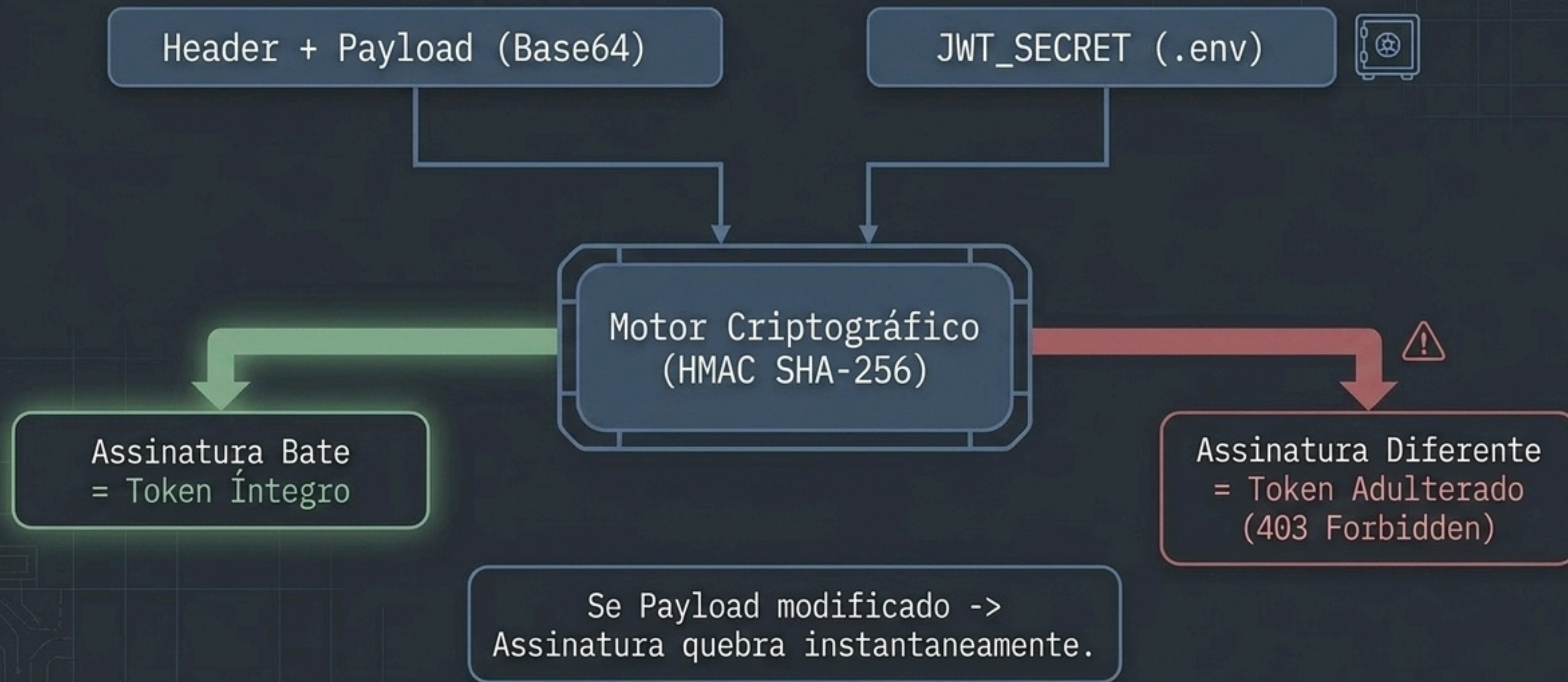
Atenção: Base64 é facilmente decodificável (ex: via jwt.io). Nunca armazene senhas aqui.

Signature

Sf1KxwRJSMekKF2QT4fwpMeJf36P0k6yJV_adQssw5c

A Matemática da Defesa: A Assinatura Digital

Como o servidor detecta a adulteração de um token sem consultar o banco de dados.



Diretrizes da Indústria: Implementando a RFC 8725

Três regras arquiteturais para emissão segura de tokens.

Algoritmo Forte

Bloqueio do ataque 
None.

Forçar uso de algoritmos robustos como

HS256 (Simétrico) ou 

RS256 (Assimétrico). 

Expiração Curta (exp)

Minimização da janela de ataque. Tokens não duram para sempre.

Configuração recomendada:
15 a 30 minutos.



O Códigog: A Recepção (Gerando o JWT)

Emitindo o token na rota de login após a validação criptográfica.

Segredo injetado em tempo de execução.

```
const token = jwt.sign(  
  { id: usuario.id },  
  process.env.JWT_SECRET,  
  { expiresIn: '15m', algorithm: 'HS256' }  
);  
res.json({ message: 'Acesso Liberado', token });
```

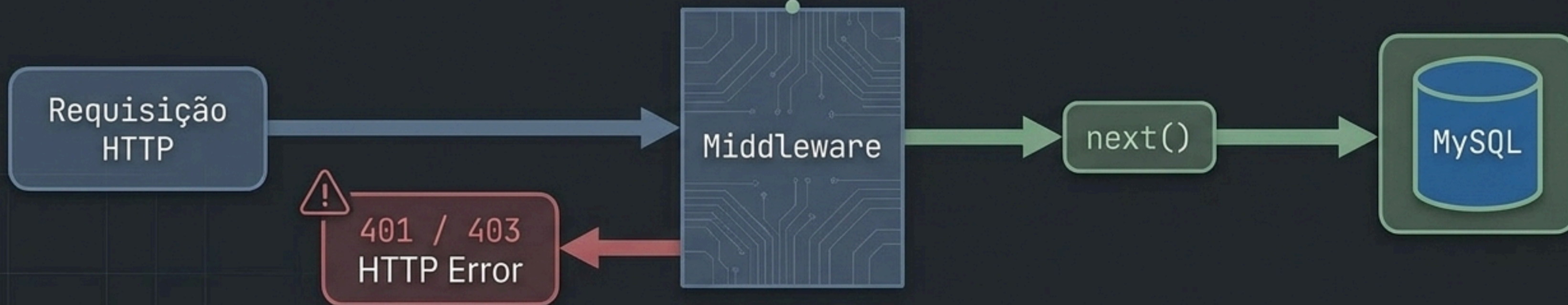
Apenas dados de identificação necessários.

Ciclo de vida curto configurado.

O Código: O Segurança da Porta (Middleware)

Interceptando a requisição antes que ela atinja os dados sensíveis.

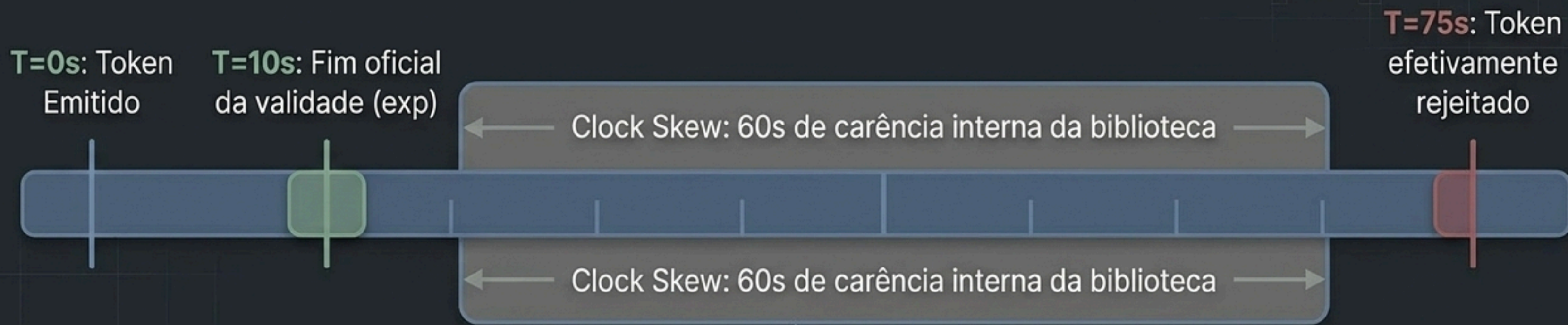
```
jwt.verify(token, process.env.JWT_SECRET)
```



O Middleware atua como um filtro global para proteger rotas de consulta (ex: `/api/notes`).

A Armadilha do Clock Skew (Tolerância de Relógio)

Por que um token de 10 segundos ainda funciona após 30 segundos?



```
jwt.verify(token, secret, { clockTolerance: 0 })
```

Desativa o recurso fantasma para testes de penetração cirúrgicos.

Matriz de Testes Táticos (Pentest Local)

Comportamentos esperados durante a auditoria da aplicação.

Ação do Atacante/Usuário	Resposta Esperada do Servidor	Status HTTP
Login bem-sucedido	Token gerado e retornado	200 OK
Requisição anônima (Sem Token)	Acesso negado pelo Middleware	401 Unauthorized
Token adulterado via Console (ex: add A no final)	Assinatura inválida detectada	403 Forbidden
Acesso com token expirado (Aguardar > 15s)	Rejeição por limite de tempo	403 / 401

Auditoria Final: Blueprint de Entrega Segura

Checklist arquitetural para o Gerenciador de Notas.

- [] Hashing na Entrada: Senhas protegidas com `Bcrypt` no cadastro.
- [] Emissão de JWT (RFC 8725): Tokens com algoritmo forte e expiração (15m).
- [] Defesa em Camadas: `Middleware` interceptando todas as rotas de acesso ao banco.
- [] Gestão de Segredos: `JWT_SECRET` e credenciais do `MySQL` estritamente no `.env`.

Zero Vulnerabilidades de Autenticação

Passo 1

Primeiro baixamos a biblioteca jsonwebtoken utilizando o seguinte comando no terminal:

```
npm install jsonwebtoken
```

Passo 2

Atualizamos o server.js importando a biblioteca no topo do arquivo:

JavaScript



```
const jwt = require('jsonwebtoken');
```

```
// IMPORTANTE: Em um sistema real, isso fica em um arquivo .env invisível!
```

```
const JWT_SECRET = 'MinhaChaveSuperSecretaEComplicada123!';
```

Passo 3

Código para gera o token Substitua o bloco de "Acesso Liberado" dentro da sua rota /login (logo após o bcrypt.compare dar verdadeiro) por isto:

```
if (senhaValida) {  
  console.log("🔓 STATUS: Acesso Liberado. Gerando JWT...");  
  
  // Aplicação da RFC 8725: Payload mínimo + Expiração + Algoritmo Fixo  
  const token = jwt.sign(  
    { id: usuarioBanco.id, usuario: usuarioBanco.usuario }, // Payload (Carga Útil)  
    JWT_SECRET, // A chave que assina  
    {  
      expiresIn: '1h', // Expira em 1 hora (Regra de Ouro)  
      algorithm: 'HS256' // Trava o algoritmo para evitar falhas de rebaixamento  
    }  
  );  
  
  // Agora enviamos o token de volta para o navegador  
  res.json({  
    message: "Acesso concedido!",  
    usuarioNoBanco: usuarioBanco.usuario,  
    token: token  
  });  
}
```

Passo 4

Agora precisamos de uma função que verifique o token antes de deixar alguém ver uma nota. Adicione este código antes da sua rota de notas:

```
// Middleware de Autenticação JWT
function autenticarToken(req, res, next) {
  // O token geralmente vem no cabeçalho HTTP: "Authorization: Bearer <token>"
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1];

  if (!token) {
    return res.status(401).json({ message: "Acesso Negado: Token não fornecido." });
  }

  // Verifica se o token é válido e se foi assinado com o nosso segredo
  jwt.verify(token, JWT_SECRET, { algorithms: ['HS256'] }, (err, usuarioDecodificado) => {
    if (err) {
      // Se expirou ou foi fraudado, cai aqui
      return res.status(403).json({ message: "Acesso Negado: Token inválido ou expirado." });
    }

    // Se deu tudo certo, guarda os dados do usuário na requisição e permite passar
    req.usuarioLogado = usuarioDecodificado;
    next(); // "Pode abrir a porta e executar a rota solicitada"
  });
}
```

Passo 5

Finalmente, adicione a palavra `autenticarToken` no meio da sua rota de buscar notas. Isso força a rota a passar pela segurança primeiro.

```
// Rota de Notas agora está PROTEGIDA
app.get('/api/notes/:id', autenticarToken, (req, res) => {
  // Para ver no console quem está tentando acessar:
  console.log(`Usuário [${req.usuarioLogado.usuario}] está tentando ler a nota ${req.params.id}`);

  // ... (o resto do seu código de consulta SQL continua igual)
});
```

Passo 6

mudanças no arquivo index.html:

adicionamos o:

```
let tokenUsuario = "";
```

o intuito dessa variável é salvar o token após o login do usuário para ele ser usado nas outras funções do site para conceder ou não acesso.

Passo 7

basicamente atualizar o index.html para fazer ele usar o tokenUsuario, porém é muito código então aqui vai ficar só no github mesmo o código não cabe no slide